
ApparelPro

Architecture Reference
for the Development Team

3-Tier Architecture · Deployment · Tech Stack · Codebase Metrics

September 2026 · Internal Document

Three-Tier Architecture

ApparelPro follows a strict three-tier architecture separating presentation, business logic, and data access. Each tier is a distinct project within the solution, enforcing clean boundaries through AutoMapper-based DTO transformations at every layer crossing.

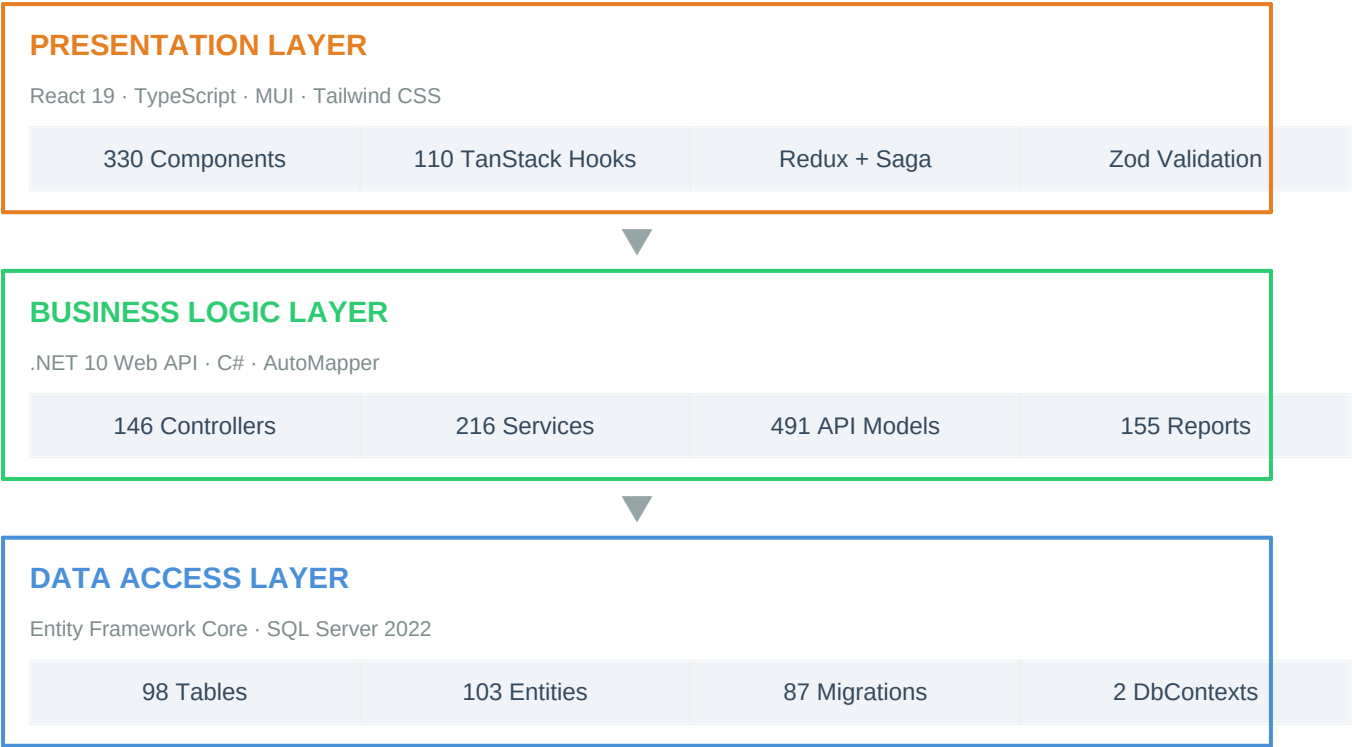


Figure 1: Three-tier architecture with codebase metrics per layer

Solution Structure

Project	Type	Role	Files
ApparelPro.WebApi	.NET 10 Web API	Controllers, API Models, Reports	647
apparelPro.BusinessLogic	.NET 10 Class Library	Services (interfaces + impl), Reports	853
ApparelPro.Data	.NET 10 Class Library	EF Core DbContext, Entities, Configs, Migrations	380
ApparelPro.Shared	.NET 10 Class Library	Constants, Extensions (pagination)	5

Request / Response Data Flow

Ingress: API Model (Controller) → Service Model (Business) → Data Model (EF Core Entity)

Egress: Data Model → Service Model → API Model → JSON Response

AutoMapper profiles enforce strict layer separation:

- **Service to API Model Mappings** — Controller ↔ Service layer transformations

- **DatabaseToServiceMappings** — Service ↔ Data layer transformations

All mappings use `.MaxDepth(2)` to prevent deep-graph traversal.

Deployment Architecture

ApparelPro is deployed on a self-hosted Ubuntu server using Coolify for container orchestration and Cloudflare Tunnel for secure external access without exposing any ports to the public internet.

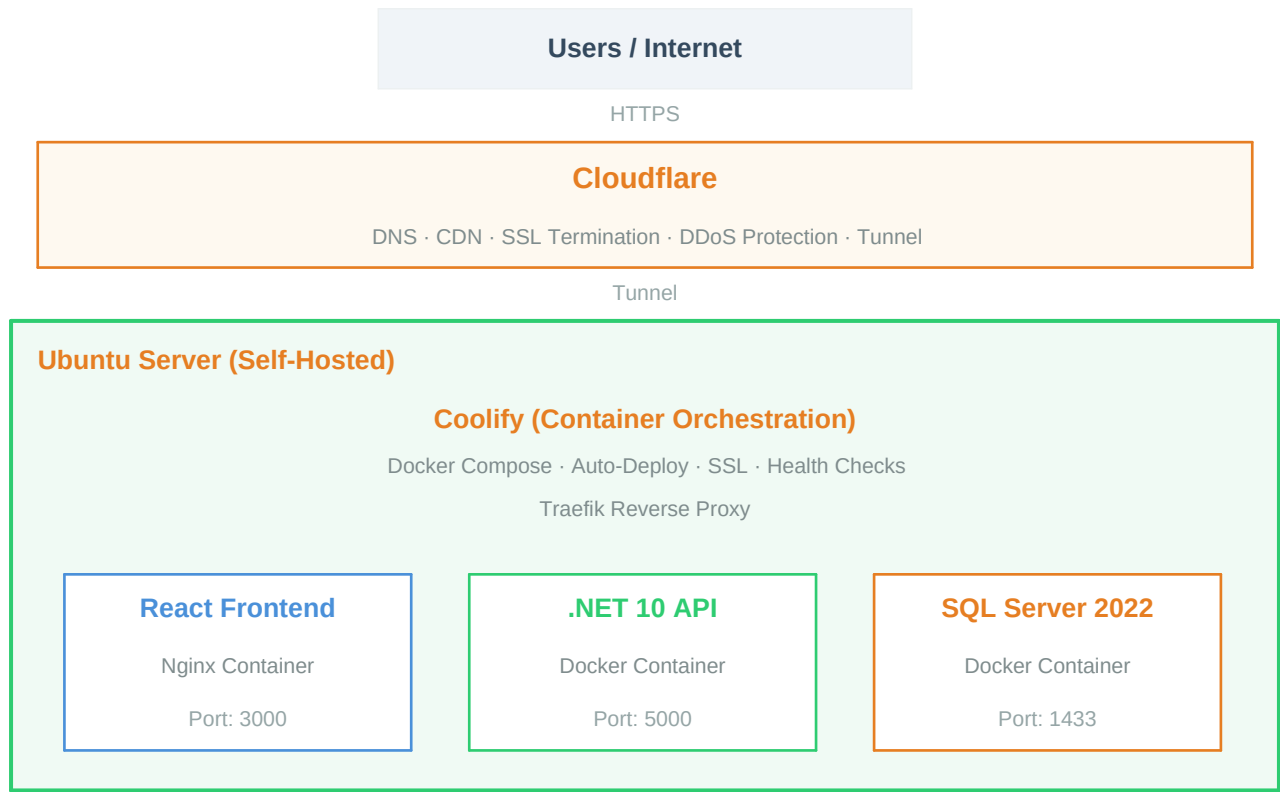


Figure 2: Production deployment — Cloudflare → Coolify → Docker containers

Deployment Components

Component	Technology	Purpose
Edge Security	Cloudflare	DNS, CDN, SSL termination, DDoS protection, WAF
Secure Tunnel	cloudflared	Encrypted tunnel from Cloudflare to server (no open ports)
Orchestration	Coolify	Docker Compose management, auto-deploy, health checks
Reverse Proxy	Traefik	Route requests to correct container by hostname
Frontend	Nginx + React	Static file serving for the SPA build
Backend API	.NET 10 Docker	Kestrel web server running the Web API
Database	SQL Server 2022	Docker container with persistent volume mount

Technology Stack

Frontend Stack

Category	Technology	Version	Purpose
Core	React	19.2.6	Component framework
Core	TypeScript	~6.0.2	Type safety
Build	Vite	8.0.12	Build tool and dev server
UI Library	MUI (Material UI)	9.0.1	Component library
Styling	Tailwind CSS	4.3.0	Utility-first CSS
Styling	Emotion	11.14.0	CSS-in-JS for MUI
State	Redux Toolkit	2.12.0	Global state management
State	Redux Saga	1.5.0	Async side effects
Server State	TanStack Query	5.100.14	API data fetching and caching
HTTP	Axios	1.16.1	HTTP client
Tables	Material React Table	3.2.1	Data grid with server-side ops
Validation	Zod	4.4.3	Runtime schema validation
Routing	React Router	7.16.0	Client-side routing
Feedback	React Toastify	11.1.0	Toast notifications

Backend Stack

Category	Technology	Purpose
Framework	.NET 10 Web API (C#)	RESTful API framework
ORM	Entity Framework Core	Database access and migrations
Database	SQL Server 2022	Relational data store
Mapping	AutoMapper	Layer-to-layer DTO transformations
Auth	JWT Bearer + MS Identity	Token authentication and user management
Deployment	Docker + Coolify	Containerized deployment
Edge	Cloudflare Tunnel	Secure external access